Technical Guidance: Enhancing Open Range Breakout (ORB) with Smart Money Concepts (SMC) and Liquidity Principles

Introduction

This document provides comprehensive technical guidance on how to evolve a classical Open Range Breakout (ORB) trading strategy by integrating advanced Smart Money Concepts (SMC) and liquidity principles. The goal is to create a more robust and precise trading system that aligns with institutional trading methodologies, aiming to improve entry accuracy, risk management, and overall profitability. This guide will cover the theoretical underpinnings, practical application, and MQL5 code examples for implementation within the MetaTrader 5 platform.

1. Synthesis of Open Range Breakout (ORB) and Smart Money Concepts (SMC)

1.1 Open Range Breakout (ORB) Fundamentals

The Open Range Breakout (ORB) strategy is a widely used day trading technique that capitalizes on the initial price action at the start of a trading session. The core premise is that the first few minutes or hours of trading often establish a range, and a decisive break above or below this range can signal the direction of the market for the rest of the day. Traders typically define an

opening range (e.g., the first 15, 30, or 60 minutes) and look for price to break out of this range. A breakout above the high of the opening range typically triggers a long entry, while a breakout below the low triggers a short entry. Stop-losses are often placed at the opposite end of the opening range, and profit targets are determined by various methods, such as fixed risk-reward ratios or subsequent support/resistance levels.

Key characteristics of ORB include:

- **Simplicity:** The rules are generally straightforward, making it accessible to many traders.

- **Volatility Capture:** It aims to capture momentum generated by early session volatility.
- **Time-Sensitive:** The strategy is highly dependent on the initial period of the trading day.
- **Defined Risk:** Stop-loss levels are clearly defined by the opening range.

However, traditional ORB strategies can suffer from false breakouts, whipsaws, and a lack of context regarding underlying market dynamics. This is where the integration of SMC and liquidity principles becomes crucial.

1.2 Smart Money Concepts (SMC) and Liquidity Principles

Smart Money Concepts (SMC) represent a trading methodology that seeks to understand and align with the actions of large institutional players, often referred to as "smart money." Unlike retail traders who typically react to price movements, smart money is believed to influence price through their substantial order flow. SMC focuses on identifying footprints left by these institutions, such as order blocks, fair value gaps (FVG), and liquidity grabs.

Liquidity is a central theme in SMC. In financial markets, liquidity refers to the ease with which an asset can be converted into cash without affecting its market price. From an SMC perspective, liquidity is often viewed as pools of pending orders (e.g., stop-loss orders, pending buy/sell orders) that accumulate at specific price levels. These liquidity pools act as targets for smart money, as their large orders require sufficient counter-party liquidity to be filled without significant slippage. When smart money pushes price towards these liquidity pools, it's often referred to as a "liquidity hunt" or "liquidity sweep."

Key SMC and liquidity concepts relevant to enhancing ORB include:

- **Order Blocks (OB):** These are specific candles or groups of candles where smart money is believed to have placed significant orders, leading to a strong directional move. They often represent areas of institutional accumulation or distribution.
- **Fair Value Gaps (FVG) / Imbalances:** These are inefficiencies in price delivery, typically seen as gaps where price moved quickly in one direction, leaving an imbalance between buyers and sellers. Smart money often re-visits these areas to

fill the imbalance before continuing the trend. * **Liquidity Grabs/Sweeps:** These occur when price briefly moves beyond a significant high or low (where liquidity is likely resting) to trigger stop-loss orders or fill pending orders, only to reverse direction shortly after. This is a classic smart money maneuver to gather liquidity before a true directional move. * **Breaker Blocks:** Similar to order blocks, but formed after a liquidity grab and a subsequent break of structure, indicating a shift in market sentiment. * **Market**

Structure Shift (MSS) / Change of Character (CoC): A break of a significant swing high or low, indicating a potential reversal or continuation of the trend, often after a liquidity grab.

By combining the time-sensitive nature of ORB with the contextual understanding provided by SMC and liquidity, traders can filter out false breakouts, identify higher-probability trading opportunities, and align their trades with the flow of institutional capital.

2. Integrating SMC and Liquidity into the ORB Strategy

The integration of SMC and liquidity principles transforms the traditional ORB strategy from a simple breakout system into a more sophisticated, context-aware approach. The core idea is to use SMC concepts to validate ORB signals and to identify optimal entry, exit, and stop-loss levels that are aligned with institutional activity.

2.1 Defining Liquidity Application in ORB

In the context of an SMC-enhanced ORB, liquidity principles are applied in several ways:

1. **Identifying Liquidity Pools around the Opening Range:** Instead of just looking at the high and low of the opening range as simple resistance/support, we view them as potential liquidity pools. Stop-loss orders from early retail traders often cluster just outside these levels. Smart money may intentionally push price to these levels to trigger these stops, providing the necessary liquidity for their larger positions.
2. **Filtering Breakouts with Liquidity Sweeps:** A common pitfall of traditional ORB is false breakouts. By incorporating liquidity sweeps, we can differentiate between a genuine breakout and a liquidity grab. A true breakout, from an SMC perspective, would ideally occur after a liquidity sweep in the opposite direction, indicating that smart money has collected the necessary orders and is now ready to move price in the intended direction.
3. **Targeting Liquidity for Profit Taking:** Just as smart money hunts for liquidity, traders can use identified liquidity pools as potential profit targets. For instance, if an ORB breakout occurs, and there's a clear liquidity pool (e.g., previous swing high/low, equal highs/lows) further in the direction of the breakout, this can serve as a logical take-profit level.
4. **Placing Stop-Losses Beyond Liquidity:** Instead of placing stop-losses just outside the opening range, SMC principles suggest placing them beyond significant liquidity areas or within protected zones (e.g., behind an order block that initiated the move). This reduces the likelihood of being stopped out by a liquidity grab.

2.2 Specific SMC Elements for ORB Refinement

2.2.1 Order Blocks (OB) and ORB

Order Blocks are crucial for identifying areas where institutional orders were placed. When integrating with ORB:

- **Confirmation of Breakout:** An ORB breakout that originates from or retests an existing bullish (for long) or bearish (for short) order block within the opening range or just before the breakout can provide strong confirmation. This suggests that the breakout is supported by institutional interest.
- **Entry Refinement:** Instead of entering immediately on the breakout, traders can wait for a retest of a valid order block that forms part of the breakout structure. This offers a lower-risk entry with a tighter stop-loss.
- **Stop-Loss Placement:** The low of a bullish order block or the high of a bearish order block can serve as a protected stop-loss level, as smart money is likely to defend these areas.

2.2.2 Fair Value Gaps (FVG) / Imbalances and ORB

FVGs represent price inefficiencies that smart money often seeks to fill. Their relevance to ORB includes:

- **Confirmation of Direction:** If an ORB breakout occurs in the direction of an existing FVG, it suggests that price is likely to move to fill that imbalance. This provides a directional bias and potential profit target.
- **Retracement Opportunities:** After an ORB breakout, price may retrace to fill a newly created FVG before continuing in the breakout direction. This retracement can offer a second-chance entry opportunity.
- **Invalidation:** If an ORB breakout moves against a significant FVG that is likely to be filled, it might indicate a false breakout or a liquidity grab.

2.2.3 Liquidity Grabs/Sweeps and ORB

Understanding liquidity grabs is paramount for avoiding false signals:

- **Pre-Breakout Liquidity Sweep:** A powerful confirmation for an ORB breakout is a liquidity sweep in the opposite direction just before the breakout. For example, before a bullish ORB breakout, price might briefly dip below the opening range low (sweeping sell-side liquidity) before reversing sharply and breaking out to the upside. This indicates that smart money has cleared out opposing orders and is ready for the true move.

- **False Breakout Identification:** If price breaks out of the opening range but immediately reverses after sweeping a clear liquidity pool, it's likely a liquidity grab and not a genuine breakout. This helps avoid entering losing trades.

2.2.4 Market Structure Shift (MSS) / Change of Character (CoC) and ORB

MSS/CoC provides context on the overall trend and potential reversals:

- **Trend Alignment:** An ORB breakout that aligns with a recent MSS/CoC in the same direction (e.g., a bullish ORB breakout after a bullish MSS) strengthens the conviction of the trade.
- **Reversal Confirmation:** If an ORB breakout occurs after a significant MSS/CoC that signals a reversal, it can be a high-probability trade setup, as it combines the initial momentum of the ORB with a broader market structure shift.

2.3 Proposed Entry, Exit, and Stop-Loss Rules for SMC-Enhanced ORB

Here's a framework for integrating these concepts into actionable trading rules:

Define Opening Range: Determine the high and low of the first X minutes/candles of the trading session (e.g., 15-minute ORB).

Entry Conditions (Long Example):

1. **Price breaks above the Opening Range High.**
2. **Confirmation of Liquidity Sweep (Optional but Recommended):** Ideally, prior to the breakout, price should have swept liquidity below the Opening Range Low or a significant previous low.
3. **Order Block Confluence:** The breakout should originate from or retest a valid bullish Order Block (formed on a higher timeframe or within the opening range).
4. **FVG Confluence:** There should be a clear Fair Value Gap (FVG) above the breakout level, indicating potential for price to move higher to fill the imbalance.
5. **Market Structure Alignment:** The breakout should align with the prevailing bullish market structure or a recent bullish Market Structure Shift (MSS).

Entry Conditions (Short Example):

1. **Price breaks below the Opening Range Low.**
2. **Confirmation of Liquidity Sweep (Optional but Recommended):** Ideally, prior to the breakout, price should have swept liquidity above the Opening Range High or a significant previous high.
3. **Order Block Confluence:** The breakout should originate from or retest a valid bearish Order Block (formed on a higher timeframe or within the opening range).

4. **FVG Confluence:** There should be a clear Fair Value Gap (FVG) below the breakout level, indicating potential for price to move lower to fill the imbalance.
5. **Market Structure Alignment:** The breakout should align with the prevailing bearish market structure or a recent bearish Market Structure Shift (MSS).

Stop-Loss Placement:

- **Conservative:** Below the low of the confirming Order Block or below the low of the liquidity sweep candle (for long trades), or above the high of the confirming Order Block or above the high of the liquidity sweep candle (for short trades).
- **Aggressive:** Just below the low of the candle that initiated the breakout from the Order Block, or just above the high of the candle that initiated the breakout from the Order Block.
- **Dynamic:** Consider moving stop-loss to breakeven after price moves a certain distance in profit or after filling a partial FVG.

Take-Profit Targets:

- **Liquidity Pools:** Target obvious liquidity pools (e.g., equal highs/lows, previous swing highs/lows, untouched daily/weekly highs/lows) in the direction of the trade.
- **FVG Fill:** Target the full or partial fill of a significant FVG.
- **Risk-Reward Ratio:** Aim for a minimum 1:2 or 1:3 risk-reward ratio, but prioritize liquidity targets.
- **Partial Profits:** Consider taking partial profits at intermediate liquidity targets and letting the remainder run with a trailing stop.

This integrated framework provides a more nuanced and institutionally-aligned approach to trading the Open Range Breakout, aiming to reduce false signals and improve overall trade quality.

3. MQL5 Code Implementation

This section provides MQL5 code examples to illustrate how the SMC-enhanced ORB strategy can be implemented within a MetaTrader 5 Expert Advisor (EA). The code will focus on key components such as defining the opening range, identifying basic SMC elements, and integrating them into trading logic. Due to the complexity of fully automating all SMC nuances, these examples will serve as a foundational framework that can be further expanded and refined.

3.1 Basic MQL5 Expert Advisor Structure

First, let's establish the basic structure of an MQL5 Expert Advisor. This includes standard EA event handlers like `OnInit()`, `OnDeinit()`, and `OnTick()`.

```
//
+------------------------------------------------------------------
+
//|
SMC_Enhanced_ORB_EA.mq5 |
//|                                                    Manus
AI       |
//|
|
//
+------------------------------------------------------------------
+
#property copyright "Manus AI"
#property link      "https://www.manus.ai"
#property version   "1.00"
#property description "SMC Enhanced Open Range Breakout Expert
Advisor"

//--- Input parameters
input int      ORB_Timeframe_Minutes = 15; // Opening Range
period in minutes
input int      ORB_Start_Hour        = 9;  // Market Open Hour
(e.g., 9 for 9:00 AM)
input int      ORB_Start_Minute      =
30; // Market Open Minute (e.g., 30 for 9:30 AM)
input int      ORB_End_Hour          = 10; // Opening Range End
Hour (e.g., 10 for 10:00 AM)
input int      ORB_End_Minute        = 0;  // Opening Range End
Minute (e.g., 0 for 10:00 AM)
input double   RiskPerTrade          = 0.01; // Risk per trade
as a percentage of balance
input int      MagicNumber           = 12345; // Unique Magic
Number for trades

//--- Global variables
double         ORB_High = 0.0;
double         ORB_Low  = 0.0;
datetime       ORB_Calculated_Time = 0;
bool           trade_taken_today = false;

//
+------------------------------------------------------------------
+
//| Expert initialization
function                                   |
```

```
//
+------------------------------------------------------------------
+
int OnInit()
  {
//--- Check for valid ORB time settings
   if (ORB_Start_Hour < 0 || ORB_Start_Hour > 23 ||
ORB_End_Hour < 0 || ORB_End_Hour > 23 ||
       ORB_Start_Minute < 0 || ORB_Start_Minute > 59 ||
ORB_End_Minute < 0 || ORB_End_Minute > 59)
     {
      Print("ERROR: Invalid ORB time settings. Please check
inputs.");
      return(INIT_PARAMETERS_INCORRECT);
     }

//--- Set the chart timeframe to the ORB timeframe for easier
calculation (optional, but good practice)
//    This might not be necessary if you use iHigh/iLow with
specific timeframes
//    ChartSetSymbolPeriod(0,
ENUM_TIMEFRAMES(ORB_Timeframe_Minutes)); // This line is for
chart manipulation, not for EA logic directly

//---
   return(INIT_SUCCEEDED);
  }

//
+------------------------------------------------------------------
+
//| Expert deinitialization
function                                 |
//
+------------------------------------------------------------------
+
void OnDeinit(const int reason)
  {
//---
   Print("SMC Enhanced ORB EA Deinitialized. Reason: ", reason);
  }

//
+------------------------------------------------------------------
+
//| Expert tick
function                                                 |
//
+------------------------------------------------------------------
+
void OnTick()
  {
```

```
//--- Get current time
   MqlDateTime current_time;
   TimeCurrent(current_time);

//--- Reset trade_taken_today flag at the start of a new day
   if (current_time.hour == 0 && current_time.min == 0 && !
trade_taken_today)
     {
      trade_taken_today = false;
      ORB_High = 0.0;
      ORB_Low = 0.0;
      ORB_Calculated_Time = 0;
     }

//--- Calculate ORB if not calculated for today and within the
ORB calculation window
   if (ORB_Calculated_Time == 0 &&
       (current_time.hour > ORB_Start_Hour ||
(current_time.hour == ORB_Start_Hour && current_time.min >=
ORB_Start_Minute)) &&
       (current_time.hour < ORB_End_Hour || (current_time.hour
== ORB_End_Hour && current_time.min < ORB_End_Minute)))
     {
      CalculateORB();
     }

//--- Execute trading logic after ORB is calculated and if no
trade taken today
   if (ORB_Calculated_Time != 0 && !trade_taken_today)
     {
      ExecuteTradingLogic();
     }
  }

//
+------------------------------------------------------------------
+
//| Custom function to calculate the Opening Range Breakout
(ORB)    |
//
+------------------------------------------------------------------
+
void CalculateORB()
  {
   // This function will be implemented in detail later
   Print("Calculating ORB...");
   // For now, just set a dummy value to indicate it's
calculated
   ORB_High = SymbolInfoDouble(_Symbol, SYMBOL_ASK) + 100 *
_Point;
   ORB_Low = SymbolInfoDouble(_Symbol, SYMBOL_BID) - 100 *
_Point;
```

```
   ORB_Calculated_Time = TimeCurrent();
  }

//
+------------------------------------------------------------------
+
//| Custom function for trading
logic                                |
//
+------------------------------------------------------------------
+
void ExecuteTradingLogic()
  {
   // This function will be implemented in detail later
   Print("Executing trading logic...");
   // For now, just set trade_taken_today to true to prevent re-
entry
   trade_taken_today = true;
  }

//
+------------------------------------------------------------------
+
```

3.2 Calculating the Opening Range

Now, let's refine the `CalculateORB()` function to accurately determine the high and low of the opening range. This will involve iterating through historical bars for the specified ORB period.

```
//
+------------------------------------------------------------------
+
//| Custom function to calculate the Opening Range Breakout
(ORB)    |
//
+------------------------------------------------------------------
+
void CalculateORB()
  {
   datetime start_orb_time = D'';
   datetime end_orb_time = D'';

   // Construct the start and end time for the ORB period for
the current day
   MqlDateTime current_day_start;
   TimeCurrent(current_day_start);
   current_day_start.hour = ORB_Start_Hour;
   current_day_start.min = ORB_Start_Minute;
```

```
   current_day_start.sec = 0;
   start_orb_time = StructToTime(current_day_start);

   MqlDateTime current_day_end;
   TimeCurrent(current_day_end);
   current_day_end.hour = ORB_End_Hour;
   current_day_end.min = ORB_End_Minute;
   current_day_end.sec = 0;
   end_orb_time = StructToTime(current_day_end);

   // Ensure we are calculating for the current day's ORB
   if (TimeCurrent() < end_orb_time)
     {
      // If current time is before ORB end time, we are still
forming the ORB
      // We need to wait until the ORB period is complete to
finalize the range
      return;
     }

   // Get historical data for the ORB period
   MqlRates rates[];
   int count = CopyRates(_Symbol, _Period, start_orb_time,
end_orb_time, rates);

   if (count <= 0)
     {
      Print("ERROR: Could not get historical data for ORB
calculation. Count: ", count);
      return;
     }

   ORB_High = 0.0;
   ORB_Low  = 999999.9; // Initialize with a very high value

   for (int i = 0; i < count; i++)
     {
      if (rates[i].high > ORB_High)
         ORB_High = rates[i].high;
      if (rates[i].low < ORB_Low)
         ORB_Low = rates[i].low;
     }

   ORB_Calculated_Time = TimeCurrent(); // Mark ORB as
calculated for today
   PrintFormat("ORB Calculated for %s: High=%.5f, Low=%.5f",
TimeToString(ORB_Calculated_Time), ORB_High, ORB_Low);
  }

//
+------------------------------------------------------------------
+
```

3.3 Identifying Smart Money Concepts (SMC) Elements

Implementing SMC elements like Order Blocks, Fair Value Gaps, and Liquidity Sweeps requires more sophisticated logic. Here, we'll provide simplified functions that can be expanded upon. A full-fledged SMC implementation would involve analyzing market structure, internal/external liquidity, and more complex pattern recognition.

3.3.1 Order Block Identification (Simplified)

A simplified approach to identifying a bearish order block could be the last up-candle before a strong down move, and vice-versa for a bullish order block. For a more robust implementation, one would consider volume, displacement, and market structure breaks.

```
//
+------------------------------------------------------------------
+
//| Function to check for a simplified Bullish Order
Block           |
//| (Last down candle before a strong up
move)                        |
//
+------------------------------------------------------------------
+
bool IsBullishOrderBlock(int index, ENUM_TIMEFRAMES timeframe,
double& ob_high, double& ob_low)
  {
   MqlRates rates[];
   if (CopyRates(_Symbol, timeframe, index, 3, rates) != 3)
return false;

   // Check if the candle at index+1 (previous candle) is a down
candle
   // and the candle at index (current candle) is a strong up
candle
   // This is a very simplified check
   if (rates[1].open > rates[1].close && // Previous candle is
bearish
       rates[0].open < rates[0].close && // Current candle is
bullish
       (rates[0].close - rates[0].open) > (rates[1].high -
rates[1].low) * 1.5) // Strong bullish move
     {
      ob_high = rates[1].high;
      ob_low = rates[1].low;
      return true;
     }
   return false;
  }
```

```
//
+------------------------------------------------------------------
+
//| Function to check for a simplified Bearish Order
Block           |
//| (Last up candle before a strong down
move)                       |
//
+------------------------------------------------------------------
+
bool IsBearishOrderBlock(int index, ENUM_TIMEFRAMES timeframe,
double& ob_high, double& ob_low)
  {
   MqlRates rates[];
   if (CopyRates(_Symbol, timeframe, index, 3, rates) != 3)
return false;

   // Check if the candle at index+1 (previous candle) is an up
candle
   // and the candle at index (current candle) is a strong down
candle
   // This is a very simplified check
   if (rates[1].open < rates[1].close && // Previous candle is
bullish
       rates[0].open > rates[0].close && // Current candle is
bearish
       (rates[0].open - rates[0].close) > (rates[1].high -
rates[1].low) * 1.5) // Strong bearish move
     {
      ob_high = rates[1].high;
      ob_low = rates[1].low;
      return true;
     }
   return false;
  }
```

3.3.2 Fair Value Gap (FVG) Identification (Simplified)

A Fair Value Gap (FVG) is typically a three-candle pattern where the low of the first candle does not overlap with the high of the third candle (for a bullish FVG), or vice-versa for a bearish FVG. It represents an imbalance in price delivery.

```
//
+------------------------------------------------------------------
+
//| Function to check for a simplified Bullish Fair Value Gap
(FVG)  |
//| (Low of candle 1 > High of candle
3)                              |
```

```
//
+------------------------------------------------------------------
+
bool HasBullishFVG(int index, ENUM_TIMEFRAMES timeframe,
double& fvg_high, double& fvg_low)
  {
   MqlRates rates[];
   // Need at least 3 candles: index+2, index+1, index
   if (CopyRates(_Symbol, timeframe, index, 3, rates) != 3)
return false;

   // rates[2] is candle 1 (oldest), rates[1] is candle 2,
rates[0] is candle 3 (newest)
   if (rates[2].low > rates[0].high) // Check for gap between
candle 1 low and candle 3 high
     {
      fvg_high = rates[2].low;
      fvg_low = rates[0].high;
      return true;
     }
   return false;
  }

//
+------------------------------------------------------------------
+
//| Function to check for a simplified Bearish Fair Value Gap
(FVG)  |
//| (High of candle 1 < Low of candle
3)                              |
//
+------------------------------------------------------------------
+
bool HasBearishFVG(int index, ENUM_TIMEFRAMES timeframe,
double& fvg_high, double& fvg_low)
  {
   MqlRates rates[];
   // Need at least 3 candles: index+2, index+1, index
   if (CopyRates(_Symbol, timeframe, index, 3, rates) != 3)
return false;

   // rates[2] is candle 1 (oldest), rates[1] is candle 2,
rates[0] is candle 3 (newest)
   if (rates[2].high < rates[0].low) // Check for gap between
candle 1 high and candle 3 low
     {
      fvg_high = rates[0].low;
      fvg_low = rates[2].high;
      return true;
     }
   return false;
  }
```

3.3.3 Liquidity Sweep Identification (Simplified)

A liquidity sweep occurs when price briefly moves beyond a significant high or low (where liquidity is likely resting) and then reverses. For ORB, we're interested in sweeps around the ORB high/low.

```
//
+------------------------------------------------------------------
+
//| Function to check for a simplified Bullish Liquidity
Sweep       |
//| (Price dips below a low and then reverses
strongly)              |
//
+------------------------------------------------------------------
+
bool HasBullishLiquiditySweep(double reference_low,
ENUM_TIMEFRAMES timeframe, int lookback_bars)
  {
   MqlRates rates[];
   if (CopyRates(_Symbol, timeframe, 0, lookback_bars, rates) !
= lookback_bars) return false;

   for (int i = 0; i < lookback_bars - 1; i++)
     {
      // Check if current candle low sweeps below reference_low
      if (rates[i].low < reference_low)
        {
         // And the next candle closes significantly higher,
indicating reversal
         if (rates[i+1].close > rates[i+1].open &&
(rates[i+1].close - rates[i+1].open) > (rates[i+1].high -
rates[i+1].low) * 0.75)
           {
            return true;
           }
        }
     }
   return false;
  }

//
+------------------------------------------------------------------
+
//| Function to check for a simplified Bearish Liquidity
Sweep       |
//| (Price rises above a high and then reverses
strongly)            |
//
+------------------------------------------------------------------
```

```
+
bool HasBearishLiquiditySweep(double reference_high,
ENUM_TIMEFRAMES timeframe, int lookback_bars)
  {
   MqlRates rates[];
   if (CopyRates(_Symbol, timeframe, 0, lookback_bars, rates) !
= lookback_bars) return false;

   for (int i = 0; i < lookback_bars - 1; i++)
     {
      // Check if current candle high sweeps above
reference_high
      if (rates[i].high > reference_high)
        {
         // And the next candle closes significantly lower,
indicating reversal
         if (rates[i+1].close < rates[i+1].open &&
(rates[i+1].open - rates[i+1].close) > (rates[i+1].high -
rates[i+1].low) * 0.75)
           {
            return true;
           }
        }
     }
   return false;
  }
```

3.4 Integrating SMC into ORB Trading Logic

Now, let's modify the `ExecuteTradingLogic()` function to incorporate the SMC elements for entry confirmation, stop-loss placement, and take-profit targeting.

```
//
+-------------------------------------------------------------------
+
//| Custom function for trading
logic                                |
//
+-------------------------------------------------------------------
+
void ExecuteTradingLogic()
  {
   if (trade_taken_today) return; // Ensure only one trade per
day

   // Get current price
   double current_bid = SymbolInfoDouble(_Symbol, SYMBOL_BID);
   double current_ask = SymbolInfoDouble(_Symbol, SYMBOL_ASK);
```

```
   // Get current candle data (for SMC checks)
   MqlRates current_rates[];
   if (CopyRates(_Symbol, _Period, 0, 2, current_rates) != 2)
return; // Need current and previous candle

   double ob_high, ob_low, fvg_high, fvg_low;
   bool   smc_confirmation_long = false;
   bool   smc_confirmation_short = false;

   //--- Check for Bullish ORB Breakout
   if (current_ask > ORB_High) // Price breaks above ORB High
     {
      PrintFormat("Potential Bullish ORB Breakout detected.
Current Ask: %.5f, ORB High: %.5f", current_ask, ORB_High);

      // SMC Confirmation for Long Trade
      // 1. Check for a recent bearish liquidity sweep below
ORB_Low (optional but strong)
      bool liquidity_swept_bearish =
HasBullishLiquiditySweep(ORB_Low, _Period, 10); // Look back 10
bars
      if (liquidity_swept_bearish) Print("  - Bullish Liquidity
Sweep detected below ORB Low.");

      // 2. Check for a bullish Order Block near the breakout or
on retest
      bool bullish_ob_found = IsBullishOrderBlock(1, _Period,
ob_high, ob_low); // Check previous candle for OB
      if (bullish_ob_found)
PrintFormat("  - Bullish Order Block found: High=%.5f, Low=%.
5f", ob_high, ob_low);

      // 3. Check for a bullish FVG above current price
(potential target)
      bool bullish_fvg_found = HasBullishFVG(0, _Period,
fvg_high, fvg_low); // Check current candles for FVG
      if (bullish_fvg_found)
PrintFormat("  - Bullish FVG found: High=%.5f, Low=%.5f",
fvg_high, fvg_low);

      // Combine SMC confirmations (adjust logic based on
desired strictness)
      if (bullish_ob_found && bullish_fvg_found &&
liquidity_swept_bearish) // Example: require all three
        {
         smc_confirmation_long = true;
         Print("  - All SMC confirmations met for LONG trade.");
        }
      else if (bullish_ob_found && bullish_fvg_found) //
Example: require OB and FVG
        {
         smc_confirmation_long = true;
```

```
         Print("  - OB and FVG confirmations met for LONG
trade.");
        }
     }

   //--- Check for Bearish ORB Breakout
   if (current_bid < ORB_Low) // Price breaks below ORB Low
     {
      PrintFormat("Potential Bearish ORB Breakout detected.
Current Bid: %.5f, ORB Low: %.5f", current_bid, ORB_Low);

      // SMC Confirmation for Short Trade
      // 1. Check for a recent bullish liquidity sweep above
ORB_High (optional but strong)
      bool liquidity_swept_bullish =
HasBearishLiquiditySweep(ORB_High, _Period,
10); // Look back 10 bars
      if (liquidity_swept_bullish) Print("  - Bearish Liquidity
Sweep detected above ORB High.");

      // 2. Check for a bearish Order Block near the breakout or
on retest
      bool bearish_ob_found = IsBearishOrderBlock(1, _Period,
ob_high, ob_low); // Check previous candle for OB
      if (bearish_ob_found)
PrintFormat("  - Bearish Order Block found: High=%.5f, Low=%.
5f", ob_high, ob_low);

      // 3. Check for a bearish FVG below current price
(potential target)
      bool bearish_fvg_found = HasBearishFVG(0, _Period,
fvg_high, fvg_low); // Check current candles for FVG
      if (bearish_fvg_found)
PrintFormat("  - Bearish FVG found: High=%.5f, Low=%.5f",
fvg_high, fvg_low);

      // Combine SMC confirmations (adjust logic based on
desired strictness)
      if (bearish_ob_found && bearish_fvg_found &&
liquidity_swept_bullish) // Example: require all three
        {
         smc_confirmation_short = true;
         Print("  - All SMC confirmations met for SHORT
trade.");
        }
      else if (bearish_ob_found && bearish_fvg_found) //
Example: require OB and FVG
        {
         smc_confirmation_short = true;
         Print("  - OB and FVG confirmations met for SHORT
trade.");
        }
```

```
     }

   //--- Execute Trade if SMC confirmed
   if (smc_confirmation_long)
     {
      // Calculate lot size based on risk
      double lot_size =
NormalizeLot(AccountInfoDouble(ACCOUNT_BALANCE) *
RiskPerTrade / (SymbolInfoDouble(_Symbol,
SYMBOL_TRADE_TICK_VALUE) * (ORB_High - ORB_Low) / _Point));
      if (lot_size <= 0) lot_size = SymbolInfoDouble(_Symbol,
SYMBOL_VOLUME_MIN);

      // Define Stop Loss (e.g., below ORB Low or confirming OB
Low)
      double stop_loss = NormalizeDouble(ob_low - 5 * _Point,
_Digits); // Example: 5 points below OB Low
      if (stop_loss == 0 || stop_loss >= current_ask) stop_loss
= NormalizeDouble(ORB_Low - 5 * _Point, _Digits); // Fallback

      // Define Take Profit (e.g., at FVG High or 1:2 RR)
      double take_profit = NormalizeDouble(fvg_high,
_Digits); // Example: FVG High
      if (take_profit == 0 || take_profit <= current_ask)
take_profit = NormalizeDouble(current_ask + (current_ask -
stop_loss) * 2, _Digits); // Fallback 1:2 RR

      // Send Buy Order
      Trade.Buy(lot_size, _Symbol, current_ask, stop_loss,
take_profit, "SMC_ORB_Long");
      trade_taken_today = true;
      PrintFormat("LONG Trade Executed: Lot=%.2f, SL=%.5f, TP=%.
5f", lot_size, stop_loss, take_profit);
     }
   else if (smc_confirmation_short)
     {
      // Calculate lot size based on risk
      double lot_size =
NormalizeLot(AccountInfoDouble(ACCOUNT_BALANCE) *
RiskPerTrade / (SymbolInfoDouble(_Symbol,
SYMBOL_TRADE_TICK_VALUE) * (ORB_High - ORB_Low) / _Point));
      if (lot_size <= 0) lot_size = SymbolInfoDouble(_Symbol,
SYMBOL_VOLUME_MIN);

      // Define Stop Loss (e.g., above ORB High or confirming OB
High)
      double stop_loss = NormalizeDouble(ob_high + 5 * _Point,
_Digits); // Example: 5 points above OB High
      if (stop_loss == 0 || stop_loss <= current_bid) stop_loss
= NormalizeDouble(ORB_High + 5 * _Point, _Digits); // Fallback

      // Define Take Profit (e.g., at FVG Low or 1:2 RR)
```

```
      double take_profit = NormalizeDouble(fvg_low,
_Digits); // Example: FVG Low
      if (take_profit == 0 || take_profit >= current_bid)
take_profit = NormalizeDouble(current_bid - (stop_loss -
current_bid) * 2, _Digits); // Fallback 1:2 RR

      // Send Sell Order
      Trade.Sell(lot_size, _Symbol, current_bid, stop_loss,
take_profit, "SMC_ORB_Short");
      trade_taken_today = true;
      PrintFormat("SHORT Trade Executed: Lot=%.2f, SL=%.5f,
TP=%.5f", lot_size, stop_loss, take_profit);
     }
  }

//
+------------------------------------------------------------------
+
//| Helper function to normalize lot
size                              |
//
+------------------------------------------------------------------
+
double NormalizeLot(double lot)
  {
   double min_lot = SymbolInfoDouble(_Symbol,
SYMBOL_VOLUME_MIN);
   double max_lot = SymbolInfoDouble(_Symbol,
SYMBOL_VOLUME_MAX);
   double step_lot = SymbolInfoDouble(_Symbol,
SYMBOL_VOLUME_STEP);

   lot = MathMax(min_lot, lot);
   lot = MathMin(max_lot, lot);
   lot = MathRound(lot / step_lot) * step_lot;
   return(lot);
  }

//
+------------------------------------------------------------------
+
//| Helper function to convert MqlDateTime to
datetime               |
//
+------------------------------------------------------------------
+
datetime StructToTime(MqlDateTime& dt)
  {
   return(MqlDateTimeToTime(dt));
  }

// Need to include Trade.mqh for CTrade class
```

```
#include <Trade\Trade.mqh>
CTrade Trade; // Global CTrade object
```

3.5 Important Considerations and Further Enhancements

- **Backtesting and Optimization:** Thoroughly backtest and optimize the input parameters (ORB times, risk, SMC lookback periods, FVG/OB criteria) using historical data. This is crucial for validating the strategy's profitability and robustness.
- **Market Structure:** The simplified SMC functions do not fully account for complex market structure shifts (e.g., identifying true swing highs/lows, breaks of structure). A more advanced EA would incorporate these for better context.
- **Multi-Timeframe Analysis:** SMC often involves analyzing multiple timeframes. For example, identifying higher-timeframe order blocks or FVGs that align with the ORB breakout on a lower timeframe.
- **Session Times:** The ORB times are hardcoded. For different instruments or trading sessions (e.g., London, New York, Asian), these times would need to be adjusted or made dynamic.
- **Slippage and Spread:** Account for slippage and spread in live trading. The current code assumes ideal execution.
- **Error Handling:** Implement more robust error handling for trade operations (e.g., checking `GetLastError()` after `Trade.Buy()` / `Trade.Sell()`).
- **Visualizations:** For debugging and understanding, consider adding graphical objects to the chart to visualize the ORB, identified Order Blocks, FVGs, and liquidity sweeps.
- **Risk Management:** The `RiskPerTrade` parameter is a good start, but consider advanced risk management techniques like daily loss limits, maximum drawdown, and position sizing based on volatility.
- **AI Integration (Advanced):** As mentioned in your profile, you're interested in AI integration. The SMC confirmations (e.g., `smc_confirmation_long`, `smc_confirmation_short`) could be inputs to an external AI model (e.g., via DeepSeek API) that makes the final trading decision, adding another layer of intelligence. This would involve MQL5's `WebRequest` function to communicate with your AI endpoint.

This MQL5 code provides a solid starting point for building an SMC-enhanced ORB strategy. Remember that successful trading strategies require continuous refinement, adaptation, and rigorous testing.

Conclusion

Integrating Smart Money Concepts and liquidity principles into a classical Open Range Breakout strategy offers a powerful approach to enhance trading performance. By understanding and anticipating institutional behavior, traders can significantly improve the accuracy of their entries, optimize stop-loss placements, and identify more precise profit targets. The MQL5 code examples provided serve as a foundational blueprint for developing a robust SMC-enhanced ORB Expert Advisor. However, it is crucial to remember that successful implementation requires continuous learning, rigorous backtesting, and adaptation to evolving market conditions. The simplified SMC detection methods presented here are a starting point; advanced traders may wish to explore more sophisticated algorithms for identifying order blocks, fair value gaps, and liquidity sweeps, potentially incorporating machine learning or AI models for predictive analysis. Ultimately, aligning with the 'smart money' narrative can lead to a more informed and profitable trading journey.

References

[1] FTMO. (n.d.). How to trade Smart Money Concepts (SMC). Retrieved from https://ftmo.com/en/how-to-trade-smart-money-concepts-smc/ [2] Medium. (n.d.). Understanding Liquidity in Smart Money Trading. Retrieved from https://medium.com/coinmonks/understanding-liquidity-in-smart-money-trading-99700ba6ae73 [3] XS. (2025, June 2). Smart Money Concept: What It Means and How to Use It. Retrieved from https://www.xs.com/en/blog/smart-money-concept/ [4] Writofinance. (2024, September 28). Liquidity Sweep – SMC and ICT Trading Concept. Retrieved from https://www.writofinance.com/liquidity-sweep-smc-ict-trading/ [5] ATAS. (2025, February 10). What Is the Smart Money Concept and How Does the ICT Trading Strategy Work?. Retrieved from https://atas.net/technical-analysis/what-is-the-smart-money-concept-and-how-does-the-ict-trading-strategy-work/ [6] MindMathMoney. (2025, February 26). Smart Money Concepts: The Ultimate Guide to Trading Like Institutional Investors in 2025. Retrieved from https://www.mindmathmoney.com/articles/smart-money-concepts-the-ultimate-guide-to-trading-like-institutional-investors-in-2025 [7] Warrior Trading. (n.d.). Opening Range Breakout Trading Strategy. Retrieved from https://www.warriortrading.com/opening-range-breakout/ [8] HowToTrade. (n.d.). Opening Range Breakout (ORB) Trading Strategy. Retrieved from https://howtotrade.com/wp-content/uploads/2023/11/Opening-Range-Breakout-ORB-Trading-Strategy.pdf [9] Fluxcharts. (n.d.). Opening Range Breakout (ORB) Trading Strategy Explained. Retrieved from https://www.fluxcharts.com/articles/trading-strategies/common-strategies/opening-range-breakout/ [10] QuantConnect. (2024, December 13). Opening Range Breakout for Stocks in Play. Retrieved from https://www.quantconnect.com/research/

18444/opening-range-breakout-for-stocks-in-play/ [11] OPOFinance. (n.d.). Mastering the Opening Range Breakout Trading Strategy. Retrieved from https://blog.opofinance.com/en/opening-range-breakout/ [12] StocksToTrade. (2024, May 3). What Is the Opening Range Breakout Strategy? A Comprehensive Guide. Retrieved from https://stockstotrade.com/opening-range-breakout/ [13] Quantified Strategies. (2025, March 7). Opening Range Breakout Strategy (ORB) for Day Trading. Retrieved from https://www.quantifiedstrategies.com/opening-range-breakout-strategy/ [14] OnlyPropFirms. (2025, February 4). The 15-Minute Opening Range Breakout Strategy. Retrieved from https://onlypropfirms.com/learn/the-15-minute-opening-range-strategy/ [15] EasyLanguageMastery. (n.d.). Opening Range Breakout Trading Strategy Design and Implementation. Retrieved from https://easylanguagemastery.com/building-strategies/opening-range-breakout-trading-strategy-design-implementation/